PrajaPulse

Feature Deep-Dive Guide | SFIC 2026 Hackathon | Swachh & Sustainable Bharat

Part	Topic
Part 1	NGO & Help Center Network
Part 2	Smart Notifications & Engagement

Part 1: NGO & Help Center Network

Explained like a story

Meet Priya

Priya spots a huge garbage pile near her kids' park. She snaps a photo, and the app says: *"Community can fix this. Want backup?"* She taps **Request Help**. Here is what happens next:

1. **Green Earth Foundation** gets a ping with the photo, location and pile size.
2. They reply: *"We can bring 20 volunteers and gloves on Sunday."*
3. The app auto-creates a **Sunday cleanup event**, and neighbors sign up.
4. The cleanup happens and the NGO uploads before/after photos.
5. Priya gets points, the NGO gets a reputation boost, and the area gets clean.

> *That is the whole layer: a matchmaking system between problems and the people who solve them.*

Why it matters

Without NGO layer	With NGO layer
Citizen thinks "I can't clean this alone" and gives up	Citizen gets instant backup
NGOs hunt for problems	Problems come to them, with photos and GPS
Nobody knows which NGO is trustworthy	Verified badges, ratings, response stats

What is inside it

- **NGO profile cards:** like a Zomato restaurant page, with specialties, coverage area, rating, response time and past projects.
- **Verification:** registration, background check and insurance, so citizens can trust who shows up.
- **6 NGO categories:** waste, water testing, health, green spaces, community building, emergency.
- **3 connection scenarios:** (1) citizen requests a cleanup, (2) citizen joins an NGO training, (3) an NGO lab verifies government repairs.

How to actually build it

Do not build everything. Build the **"Request Help" loop** first.

Step 1: Three database tables (the foundation)

- **ngos:** name, category, service area (PostGIS location plus radius), contact, verified (yes/no), rating
- **help_requests:** which issue, which NGO, status (pending, accepted, done), date
- **ngo_reviews:** stars and comments from citizens

Step 2: The matching logic (simple, no AI needed)

```
Issue reported -> find NGOs where:
  - category matches (waste issue -> waste NGO)
  - issue location is inside the NGO's service area
-> sort by rating + response time -> show top 3
```

One PostGIS query does this.

Step 3: The "Request Help" button

Citizen taps it, a row goes into **help_requests**, and the NGO gets a WhatsApp/SMS/email alert (Firebase or Twilio). The NGO opens a simple web page and taps **Accept** or **Decline**.

Step 4: Auto-create the event

When the NGO accepts, the app creates a cleanup event with date, place and volunteer slots, and citizens can join.

Step 5: Close the loop

After the event, the NGO uploads an after-photo and citizens rate them. That updates the NGO's rating and awards points.

Hackathon shortcut (2-week reality check)

You do not need full verification for a demo. Do this:

1. **Seed 3-5 NGO profiles**, real local ones if you can talk to them, otherwise clearly marked demo data.
2. Build the **profile card + Request Help + Accept button**.
3. Show the **notification and auto-created event**.
4. **Describe verification as a process** on a slide (manual checklist and "Verified" badge) instead of coding it.

> *Judges care more about a smooth live demo than a perfect backend.*

Bonus ideas to make it shine

- **NGO "Capacity meter":** NGOs set "I can do 2 events this week," which avoids overload.
- **Smart NGO suggestions:** "Water Quality Alliance is 1.2 km away, and their response time is 3 hrs."
- **Impact receipts for NGOs:** auto-generated PDF of volunteers, kg collected and photos, which they can show to CSR funders. NGOs will love this, and it gives them a reason to join.
- **Real talk to NGOs:** call 2-3 local NGOs before the pitch and ask "Would you use this?" Even one quote like *"yes, this would save us hours"* is pitch gold.

Part 2: Smart Notifications & Engagement

Explained like a story

Meet Arjun

Arjun downloaded PrajaPulse, reported one garbage pile, and then forgot about the app. Like 90% of us do. Then at **7:00 AM** his phone buzzes:

> *"Your locality briefing: 3 issues need you. The sewage leak you upvoted just hit 47 votes."*

He taps. He votes. Two days later: *"Your report just hit critical priority. Sent to authorities NOW!"*

> *That is the whole layer: it brings people back at the exact moment their action matters.*

Why it matters

Most civic apps die because people report once and vanish. Notifications are the **heartbeat** that keeps the loop alive.

Without it	With it
"Did my report do anything?"	"Officer acknowledged, expected fix Friday"
Votes trickle in slowly	Neighbors get pinged, issues escalate faster
Volunteers forget the cleanup	Reminder the evening before
Users quietly churn	Weekly wins keep them proud and active

The 3 types of notifications

1. Morning Digest (7 AM)

Think Swiggy/Zomato, but for your neighborhood.

- **Urgent:** "Sewage leak needs your vote"
- **Trending:** "45 people want a cleanup Sunday"
- **Resolved:** "Park cleaned, you helped!"
- **Your impact:** "3 issues fixed, 180 points earned"

2. Real-Time Alerts

Only fire when something **actually changes**:

- Your issue crossed a threshold
- Officer acknowledged or ignored it
- A fixable issue appeared near you ("Want to organize a cleanup?")

3. Weekly Impact Report

> *"You helped resolve 2 issues. You're Silver, 45 points from Gold. Your area improved 22%. You're Top 10 in your ward!"*

The secret sauce: do not be annoying

Bad notifications get the app **uninstalled**. So the design includes:

- **Frequency control:** real-time / daily / weekly / off
- **Channel choice:** push, SMS, email, WhatsApp
- **Topic choice:** only water issues? Fine.
- **Quiet hours:** nothing after 9 PM
- **Learning mode:** ignore a type of alert a few times and the app sends fewer
- **Win-back nudges:** inactive for 2 weeks gets "3 new cleanups near you"

How to build it

Step 1: Pick your channels

- **Push:** Firebase Cloud Messaging (free, easy)
- **SMS:** Twilio or MSG91 (needed for officers and low-data users)
- **WhatsApp:** WhatsApp Business API (Phase 2)

For the hackathon: **Push + SMS only.**

Step 2: One table for preferences

```
notification_prefs:
  user_id, channel, frequency,
  topics[], quiet_start, quiet_end
```

Step 3: Event triggers (the brain)

Your backend already knows when things happen. Just hook in:

```
Issue score crosses 5  -> notify ward officer + reporter
Officer acknowledges   -> notify reporter + upvoters
Cleanup tomorrow       -> remind signed-up volunteers
New Type-A issue < 500m -> notify nearby active users
```

Step 4: The morning digest job

A **cron job at 7 AM** runs per user:

1. Query issues within their radius
2. Pick the top 3 (urgent, trending, resolved)
3. Add their weekly points
4. Send one push

Node has **node-cron**, or use Google Cloud Scheduler.

Step 5: Personalization (start dumb)

Skip ML at first. Use simple rules:

- Opened the last 3 digests? Keep sending.
- Ignored 5 in a row? Drop to weekly.
- Interested in water? Rank water issues first.

Hackathon shortcut

1. Build **2 triggers**: "your issue escalated" and "officer acknowledged."
2. Build **1 digest** that you can trigger manually with a button during the demo.
3. Show a **preferences screen** with quiet hours.
4. Demo it live: phone buzzes on stage as the vote count crosses the threshold.

> *That moment alone sells the whole platform.*

Watch out for

- **Notification fatigue:** cap at about 3 pushes per day, with critical ones exempt.
- **Language:** send in the user's language (Tamil, Hindi, English).
- **Privacy:** do not show sensitive details on the lock screen.
- **Spam from officers' side:** batch reminders so one officer does not get 40 SMS an hour.

Bonus ideas

- **Streaks:** "5 weeks in a row of civic action" (Duolingo-style)
- **Social proof:** "12 of your neighbors voted today"
- **Closing-the-loop pings:** "The garbage you reported 2 weeks ago is still clean" (builds trust)
- **Smart timing:** learn when each user opens notifications and send then, not at a fixed 7 AM